

Classificação de Estágios do Sono

Gabriel Batista Ícaro Santos Wendell Calixto

Universidade Federal de São João del-Rei (UFSJ)
Trabalho 2 — Machine Learning Aplicado à Engenharia

8 de julho de 2026

Sumário

- 1 Formulação do Problema
- 2 Preparação dos Dados
- 3 K-Nearest Neighbors
- 4 Support Vector Machines
- 5 Regressão Logística

Formulação do Problema

Objetivo

Desenvolver e comparar modelos de **classificação multiclasse** para identificar os estágios do sono (Acordado, N1, N2, N3, REM) a partir de *features* extraídas de épocas de 30 segundos de EEG.

- Base: Dados do PhysioNet (*Sleep-EDF Expanded*)
- *Features*: Estatísticas temporais + potências espectrais (28 atributos)
- Desafios principais: desbalanceamento e sobreposição entre classes

Preparação dos Dados

Preparação dos Dados

- Extração de *features* em janelas de 30 segundos
- Separação por noites inteiras (`GroupShuffleSplit`)
- Padronização com `StandardScaler`
- Tratamento do desbalanceamento (considerado nos modelos)

Separação por Noites Inteiras

Por que não usar `train_test_split`?

O `train_test_split` (mesmo com estratificação) divide as épocas de forma aleatória, misturando épocas de uma mesma noite entre treino e teste.

Problema

Épocas de uma mesma noite são altamente correlacionadas (mesmo paciente, mesma condição fisiológica). Isso causa **vazamento de informação** e uma avaliação otimista irrealista.

Solução adotada: *GroupShuffleSplit*

- Separa **noites inteiras** (chavepar) entre treino e teste
- Garante que o modelo nunca veja épocas de uma noite durante o treino
- Simula melhor a realidade clínica (generalização para novos pacientes)

Features Selecionadas para os Modelos

Critério de seleção

Mantivemos variáveis com maior poder discriminante e menor redundância entre si.

```
1  # Lista de colunas de entrada (Features).
2  features_cols = [
3      'variancia', 'assimetria', 'curtose_excesso', '
4      potencia_delta',
5      'potencia_theta', 'potencia_alpha', 'potencia_beta', '
6      relativo_delta',
7      'relativo_theta', 'relativo_alpha', 'relativo_beta', '
8      razao_alpha_theta', 'hjorth_complexidade'
9  ]
10
11 X = df[features_cols]
12 y = df['id_estagio'] # Alvo numérico para classificação
```

- Removemos `valor_rms` e outras variáveis altamente correlacionadas para melhorar a eficiência dos modelos.

K-Nearest Neighbors

Modelo K-Nearest Neighbors (KNN)

Aprendizado Baseado em Instâncias

O KNN não cria uma equação global. Ele classifica uma nova amostra comparando-a diretamente com os exemplos mais semelhantes do conjunto de treino.

- Baseado em distância (ex: Euclidiana ou Manhattan).
- Natureza não paramétrica: a complexidade cresce com o tamanho da base.
- Exige padronização dos dados.
- O hiperparâmetro k controla o equilíbrio entre viés e variância

Intuição e Funcionamento do KNN

Funcionamento

Para classificar uma **nova época de 30 segundos**:

- 1 Calcula a distância para todas as épocas do conjunto de treino
- 2 Identificação dos k vizinhos mais próximos
- 3 Realiza votação pela classe majoritária (ou ponderada por distância)

Hiperparâmetro chave

k = número de vizinhos

- k pequeno $\rightarrow$ alta variância
- k grande $\rightarrow$ alto viés

Estratégia adotada

GroupShuffleSplit com base em chavepar (paciente + noite)

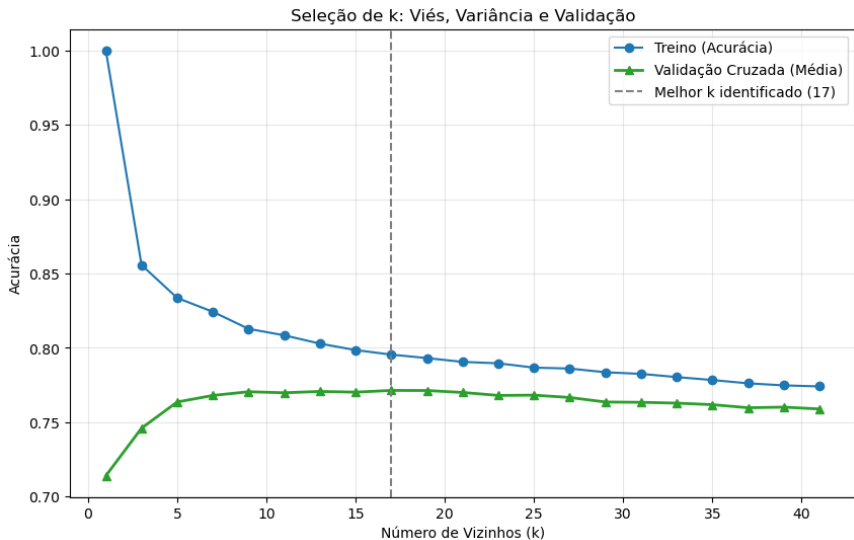
```
1 from sklearn.model_selection import GroupShuffleSplit,
   cross_val_score, StratifiedKFold
2 from sklearn.neighbors import KNeighborsClassifier
3 from sklearn.pipeline import Pipeline
4 from sklearn.preprocessing import StandardScaler
5
6 # Divisão por Noite
7 gss = GroupShuffleSplit(n_splits=1, test_size=0.30,
   random_state=42)
8
9 for train_idx, test_idx in gss.split(X, y, groups=df['
   chavepar']):
10     X_train, X_test = X.iloc[train_idx], X.iloc[test_idx]
11     y_train, y_test = y.iloc[train_idx], y.iloc[test_idx]
12     break
```

Seleção do Hiperparâmetro k

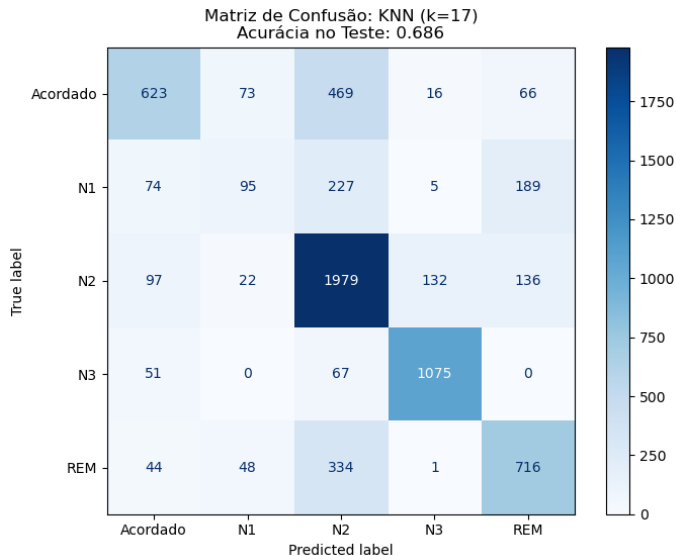
```
1  # Validação Cruzada (dentro do treino)
2  k_values = list(range(1, 42, 2))
3  cv = StratifiedKFold(n_splits=5, shuffle=True, random_state
4      =42)
5
6  for k in k_values:
7      knn_model = Pipeline([
8          ("scaler", StandardScaler()),
9          ("knn", KNeighborsClassifier(n_neighbors=k, weights='
10         uniform', metric='manhattan')),
11     ])
12
13     knn_model.fit(X_train, y_train)
14     train_scores.append(knn_model.score(X_train, y_train))
15     test_scores.append(knn_model.score(X_test, y_test))
16
17     scores = cross_val_score(knn_model, X_train, y_train, cv=
18     cv, scoring="accuracy")
19     cv_means.append(scores.mean())
```

- *Melhor k por validação cruzada: 17*

Gráfico: Seleção de k



Matriz de Confusão - KNN



Relatório de Classificação - KNN

Classe	Precisão	Recall	F1-score	Suporte
Acordado	0,701	0,500	0.583	1247
N1	0,399	0,161	0,229	590
N2	0,643	0,836	0,727	2366
N3	0,875	0,901	0,888	1193
REM	0,647	0,626	0,636	1143

Tabela 1: Relatório de classificação do KNN no conjunto de teste.

- **Melhor classe:** N3 ($F1 = 0,888$)
- **Pior classe:** N1 ($F1 = 0,229$)

Conclusão: Desempenho do Modelo KNN

- O KNN estabeleceu um ponto de partida (*baseline*), atingindo 68,6% de acurácia em um problema complexo de 5 classes.
- A lacuna de 10,92% entre treino e teste é o sinal característico de sobreajuste, refletindo a dificuldade em generalizar padrões entre diferentes pacientes.
- Desempenho crítico no estágio **N1 (Recall 16,1%)**, evidencia como a classe majoritária (N2) "vicia" a votação por distância.
- Resultados reforçam a necessidade de modelos mais robustos ao desbalanceamento.

Support Vector Machines

Configuração do SVM

Modelo avaliado

```
StandardScaler + SVC(kernel="rbf", class_weight="balanced")
```

- O SVM separa as classes procurando uma fronteira com boa margem entre elas.
- O kernel RBF foi usado porque os estágios do sono não seguem uma separação linear simples.
- O ajuste `class_weight=balanced` aumenta o peso dos erros nas classes menos frequentes.

Por que usar SVM?

Separação não linear

Os estágios do sono se sobrepõem no espaço das características. O RBF permite criar fronteiras curvas para lidar melhor com essa sobreposição.

Classes próximas

N1, N2 e REM têm padrões parecidos em algumas épocas de 30 segundos, o que torna a separação mais difícil.

Ideia do modelo

O SVM tenta manter uma margem ampla entre as classes, mas aceita alguns erros quando os dados ficam naturalmente misturados.

Etapas usadas

- 1 Separação entre treino e teste por noites inteiras com `GroupShuffleSplit`.
- 2 Padronização das variáveis com `StandardScaler`.
- 3 Busca de hiperparâmetros com `GridSearchCV` e `GroupKFold`.
- 4 Seleção do melhor modelo pelo F1-score macro.

Cuidado metodológico

A validação por grupos evita que épocas da mesma noite apareçam ao mesmo tempo no treino e no teste.

Busca de hiperparâmetros

Grade testada

A busca variou C e γ , mantendo o kernel RBF e o tratamento do desbalanceamento entre as classes.

Parâmetro	Valores testados
C	0,1, 1, 10, 100, 1000
γ	scale, 0,001, 0,01, 0,1, 1, 10
Seleção	F1-score macro

Melhor configuração

$C = 100$, $\gamma = 0,01$ e F1 macro médio de validação igual a **0,681**.

Efeito de C e γ

$C = 100$

Penaliza erros de classificação com intensidade alta, mas sem usar o ajuste mais rígido testado.

$\gamma = 0,01$

Dá uma influência intermediária às amostras, permitindo uma fronteira não linear sem torná-la local demais.

Leitura

A configuração escolhida ficou em um meio-termo: flexível o suficiente para capturar relações não lineares, mas sem prender demais a fronteira ao treino.

Métrica	Valor
Acurácia no treino	0,7897
Acurácia no teste	0,7104
F1 macro no teste	0,6692
Gap treino-teste	0,0794

Tabela 2: Resultados gerais do SVM RBF no conjunto de teste.

Leitura do resultado

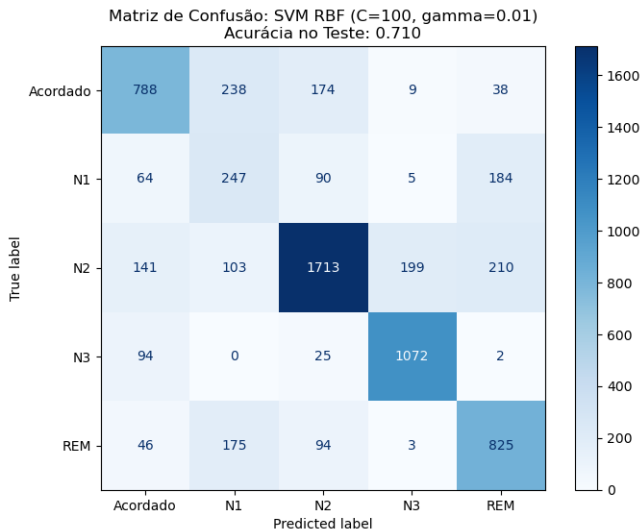
O gap de 7,94 pontos percentuais mostra uma perda moderada de generalização. Ainda assim, o teste ficou próximo do treino, então não há sinal de sobreajuste severo.

Classe	Precisão	Recall	F1-score	Suporte
Acordado	0,70	0,63	0,66	1247
N1	0,32	0,42	0,37	590
N2	0,82	0,72	0,77	2366
N3	0,83	0,90	0,86	1193
REM	0,66	0,72	0,69	1143

Tabela 3: Relatório de classificação do SVM RBF no conjunto de teste.

- Melhor classe: **N3**, com F1-score de 0,86 e recall de 0,90.
- Classe mais difícil: **N1**, com F1-score de 0,37.

Matriz de confusão do SVM



Interpretação dos resultados

N3

O SVM reconheceu melhor o sono profundo, com recall de 0,90 e F1-score de 0,86.

N1

N1 continuou sendo a classe mais instável. É uma fase de transição e se mistura com padrões de Acordado e N2.

Consequência para o modelo

O SVM melhora a separação não linear, mas a sobreposição fisiológica entre os estágios ainda limita o desempenho nas transições do sono.

Conclusão: modelo SVM

- A melhor configuração foi $C = 100$ e $\gamma = 0,01$.
- O modelo atingiu 71,04% de acurácia e F1 macro de 66,92% no teste.
- O gap de 7,94 p.p. sugere variância moderada, mas não sobreajuste severo.
- O melhor desempenho ocorreu em N3; a maior dificuldade permaneceu em N1.

Mensagem principal

O SVM RBF separou razoavelmente as classes mais definidas, mas ainda confundiu estágios de transição.

Regressão Logística